Security Review

Cantina Managed

JUPITER LEND

Cantina Managed review by:
Gerard Persoon, Lead Security Researcher
Jonatas Martins, Security Researcher

October 13, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 Medium Risk . . . 4
3.1.1 Tracked share-price inflation via rounding . . . 4
3.2 Low Risk . . . 4
3.2.1 Rounddown of fees . . . 4
3.2.2 Price change emit is not always done . . . 4
3.2.3 Withdraw remaining underlying while burning zero `xTokens` . . . 5
3.2.4 Protocol fee claims erase fractional protocol entitlement . . . 5
3.3 Informational . . . 6
3.3.1 Production and the staging environment have the same id . . . 6
3.3.2 Slippage check code is copied multiple times . . . 6
3.3.3 Difference in call to `accrue()` in fee update functions . . . 7
3.3.4 The names of update fee functions and MAX fee constants are confusing . . . 7
3.3.5 Function `transfer_protocol_authority()` isn't obvious . . . 7
3.3.6 The ProtocolConfig PDA is not always enforced . . . 8
3.3.7 Two different ways to create errors . . . 8
3.3.8 No explicit checks are done on `mints` . . . 9
3.3.9 Explicit checks on `fee_recipient` . . . 9
3.3.10 Restrictions to `ATA` not done consistently . . . 10
3.3.11 The old fee recipient doesn't get the fees at `update_partner_fee_recipient()`, . . . 10
3.3.12 Duplicate unused functions . . . 10
3.3.13 Checks that `mint` and `f_token_mint` are equal to their stored values . . . 11
3.3.14 Unnecessary `mut` s . . . 11
3.3.15 Dead code . . . 11
3.3.16 Frontrunning creating `ATA` . . . 12
3.3.17 Funds can be stuck in transit ATA . . . 12
3.3.18 No way to update the `name` / `symbol` / `uri` . . . 12
3.3.19 Functions `transfer_vault_to_partner_recipient()` and `transfer_vault_to_protocol_authority()` are very similar . . . 12
3.3.20 Names are not directly clear in struct `UnderlyingOp` . . . 13
3.3.21 6 data accounts are not explictly checked . . . 13
3.3.22 7 copies of get_lending_update_rate_cpi_accounts . . . 13
3.3.23 `*_SEED` in different files . . . 13
3.3.24 The `EXCHANGE_PRICES_PRECISION` is hardcoded . . . 14
3.3.25 `GOVERNANCE_MS` is the same as in the underlying `fluid` protocol . . . 14
3.3.26 Meaning of `user_f_token_balance` and `protocol_f_token_balance` is not clear 14
3.3.27 Variable `last_f_token_price` has no comment about scaling . . . 14
3.3.28 `partner_f_token_balance()` and `estimatePartnerClaimable()` behave differently . . . 15
3.3.29 Error `WrapperVaultBalanceUnderrun` not used . . . 15
3.3.30 Both error ranges overlap . . . 15
3.3.31 Parameters for `pub struct LogAccrue` are confusing . . . 16
3.3.32 The comment for function `calc_total_delta()` is different than the implementation . . . 16
3.3.33 `partner_delta_scaled` has different meaning in different functions . . . 16
3.3.34 Failing `update_rate()` blocks all functions . . . 16
3.3.35 Pause functionality . . . 17

3.3.36 Calculation of `maintainer_fee` . . . 17
3.3.37 No zero check `x_to_mint` in `mint_underlying()` . . . 17
3.3.38 Inversion of the `fToken` price not 100% accurate in `mint_underlying()` . . . 18
3.3.39 Name `withdraw_to_f_token()` not consistent . . . 18
3.3.40 `x_supply` could be 0 . . . 18
3.3.41 Comment in function `div()` is not accurate . . . 19
3.3.42 Calculation in test `ftokens_burned_for_withdraw()` is incorrect . . . 19
3.3.43 Accrual events omit wrapper and token mint . . . 19

1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.
Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Aug 24th to Sep 1st the Cantina team conducted a review of lend-referral on commit hash f57d29e0. The team identified a total of **48** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	1	1	0
Low Risk	4	3	1
Gas Optimizations	0	0	0
Informational	43	18	25
Total	**48**	**22**	**26**

2.1 Scope

The security review had the following components in scope for lend-referral on commit hash f57d29e0:

```
├── crates
│   └── library
│       └── src
│           ├── errors.rs
│           ├── lib.rs
│           └── math
│               ├── casting.rs
│               ├── mod.rs
│               ├── safe_math.rs
│               └── u256.rs
└── programs
    └── lending_referral
        └── src
            ├── constant.rs
            ├── errors.rs
            ├── events.rs
            ├── helper.rs
            ├── lib.rs
            ├── invokes
            │   ├── lending.rs
            │   └── mod.rs
            ├── module
            │   ├── admin.rs
            │   ├── mod.rs
            │   └── user.rs
            ├── state
            │   ├── context.rs
            │   ├── mod.rs
            │   ├── seeds.rs
            │   └── state.rs
            └── utils
                ├── deposit.rs
                ├── mod.rs
                └── withdraw.rs
```

3 Findings

3.1 Medium Risk

3.1.1 Tracked share-price inflation via rounding

Severity: Medium Risk.

Context: deposit.rs#L79, deposit.rs#L98, deposit.rs#L141, deposit.rs#L173-L178, withdraw.rs#L18, withdraw.rs#L35-L39, withdraw.rs#L80, withdraw.rs#L94-L98.

Description: The round downs in `deposit_f_token()`, `deposit_underlying()`, `withdraw_to_f_token()` and `redeem_to_underlying()` can lead to share-price inflation (similar to a donation attack) because the rounded down values are donated to everyone else in the pool.

If there is a low TVL then this allows an attack with the following recipe: shrink the pool down to one share you own, inflate that share's price through a few cheap self-trades, wait for someone to deposit a normal-sized amount into the now-inflated price, then cash out your share for a disproportionate slice of what they just put in.

It scales with the victim's deposit size, gross theft can approach roughly a quarter of whatever the victim deposits.

Recommendation: A potential solution to prevent this is to use dead shares. Alternatively a first deposit can be done to prevent this issue.

```
+   pub const MIN_LOCKED_SHARES: u64 = 1_000;

  pub fn deposit_underlying(...) ... {
  //...
      let x_to_mint = if x_supply == 0 {
-         gross_f_shares
+         require!(gross_f_shares > MIN_LOCKED_SHARES,
↪  ErrorCodes::WrapperInitialDepositTooSmall);
          gross_f_shares - MIN_LOCKED_SHARES
      } else {
  //...
  }
```

The same should be done in `deposit_f_token()`.

For `mint_underlying()`, the floor must be seeded by `deposit_f_token()` / `deposit_underlying()`, not the exact-output:

```
  pub fn mint_underlying(...)... {
  //...
      let x_supply = ctx.accounts.wrapper_config.total_x_shares;
+     require!(x_supply != 0, ErrorCodes::WrapperInitialDepositTooSmall);
  //...
  }
```

Jupiter: Fixed in PR 8.

Cantina Managed: Fix verified.

3.2 Low Risk

3.2.1 Rounddown of fees

Severity: Low Risk.

Context: helper.rs#L25-L41, deposit.rs#L19-L42.

Description: Function `calc_total_delta()` has two rounddowns, which can cause the fees to round down to 0 if `accrue()` is called very often.

Jupiter: Acknowledged, prevented mostly via internal scaling.

Cantina Managed: Acknowledged.

Recommendation: Consider storing fractional fees.

3.2.2 Price change emit is not always done

Severity: Low Risk.

Context: helper.rs#L25-L41, deposit.rs#L19-L75.

Description: In the following situations `total_delta_scaled == 0` and there is an early return in function `accrue()`:

1) When `partner_fee_bps == 0`.
2) When a fee is rounded away to dust.

In both situations the price could have moved but no `emit!` is done. This means the `emit!` trail isn't complete.

Jupiter: Fixed in commit e3d4065.

Cantina Managed: Fix verified.

Recommendation: Consider always doing an emit when the price has changed.

3.2.3 Withdraw remaining underlying while burning zero `xTokens`

Severity: Low Risk.

Context: withdraw.rs#L149-L183.

Description: In `withdraw_to_underlying()`, the shares burned are:

- `x_to_burn = ceil(f_to_burn_scaled · total_x_shares / user_f_token_balance)`

A small exact-output withdrawal can ceil up to the entire remaining share supply while debiting only part of the backing. This will leave `total_x_shares == 0` with a still-positive `user_f_token_balance`.

Once in that orphaned state, `ceil(f_to_burn_scaled · 0 / user_f_token_balance) = 0`. So any account can withdraw the remaining underlying (which will only be a small amount) while burning zero `xTokens`.

Jupiter: Fixed in commit e3d4065.

Cantina Managed: Fix verified.

Note: fixing this would be useful because this breaks an invariant.

Recommendation: This could be solved in the following way:

```
+ if x_to_burn == 0 {
+    return Err(ErrorCodes::WrapperZeroShares.into());
+ }
```

Possibly check if there is a full `f_token` left when the last `x_tokens` are used and then revert. The user should then use `redeem_to_underlying()` to retrieve all remaining value.

```
+ if x_to_burn == x_supply
+        && user_f_balance_scaled.safe_sub(f_to_burn_scaled)? >= INTERNAL_SCALE
+    {
+        return Err(ErrorCodes::WrapperExactWithdrawStrandsShares.into());
+    }
```

3.2.4 Protocol fee claims erase fractional protocol entitlement

Severity: Low Risk.

Context: withdraw.rs#L300-L310.

Description: `WrapperConfig.protocol_f_token_balance` is stored as a scaled `fToken` amount, where one raw `fToken` equals `INTERNAL_SCALE` (1,000,000) accounting units. This precision is intended to preserve fee amounts smaller than one transferable token.

However, `claim_protocol_fees` subtracts the complete scaled claim amount from the protocol balance and only afterwards converts it to whole raw `fTokens` for the transfer. A claim of 10.5 `fTokens` therefore clears 10.5 `fTokens` from the storage but transfers only 10 raw `fTokens`. A claim of 0.5 `fTokens` clears the storage and transfers zero tokens.

Additionally, the program's fee-claim functions use different units. `claim_protocol_fees` accepts its `amount` argument in scaled `fToken` units, while `claim_partner_fees` accepts its `amount` argument in raw `fToken` units. Therefore, the same numeric amount has a different meaning in each function. For example, 10 raw `fTokens` must be represented as `10 * INTERNAL_SCALE` in `claim_protocol_fees`, but as `10` in `claim_partner_fees`.

Jupiter: Fixed in commit 2f64862.

Cantina Managed: Fix verified.

This mismatch makes the two fee-claim interfaces inconsistent and can cause a caller to provide an amount in the wrong unit.

For example, assume the vault contains 10 `fTokens`, with 9 `fTokens` tracked for users, 0.5 `fTokens` tracked for the protocol, and a 0.5-`fToken` partner residual. When the protocol authority claims all fees, the contract sets the protocol balance to zero and transfers zero tokens. The partner residual is then recalculated as:

```
10 vault fTokens - 9 user fTokens - 0 protocol fTokens = 1 partner fToken
```

The partner can consequently claim one whole `fToken`, even though only 0.5 `fTokens` were previously attributable to the partner. The protocol's 0.5-`fToken` entitlement has been erased and combined with the existing partner remainder. If the residual does not immediately round up to a whole token, the amount remains in the vault but is no longer recorded as protocol-owned, so it may become untracked or be allocated to the partner by later rounding.

Recommendation: Use raw `fToken` units consistently for both functions, matching the existing denomination used by `claim_partner_fees`. Change `claim_protocol_fees` to accept and validate a raw amount.

3.3 Informational

3.3.1 Production and the staging environment have the same id

Severity: Informational.

Context: lib.rs#L15-L19.

Description: Both the production and the staging environment have the same id configured. This could result in mistakes.

Jupiter: Already aware and production one will be different.

Cantina Managed: Acknowledged.

Recommendation: Use different ids.

3.3.2 Slippage check code is copied multiple times

Severity: Informational.

Context: user.rs#L43-L45, user.rs#L59-L61, user.rs#L77-L79, user.rs#L101-L103.

Description: The slippage check code is copied multiple times, which makes maintenance more difficult.

Recommendation: Consider moving this check to a small function, for example in the following way:

```
fn enforce_min_out(actual: u64, min: u64) -> Result<u64> {
    if actual < min {
        return Err(ErrorCodes::WrapperSlippageExceeded.into());
    }
    Ok(actual)
}
```

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.3 Difference in call to `accrue()` in fee update functions

Severity: Informational.

Context: admin.rs#L37-L48, admin.rs#L133-L151, deposit.rs#L19-L42.

Description: Function `update_partner_fee()` calls `accrue()`, which is relevant because `accrue()` uses the fees.

However the comparable function `update_protocol_yield_fee()` doesn't do this. Most likely because it would have to be called for all wrappers, which wouldn't fit in the transaction size.

Recommendation: Consider calling `accrue()` for all the wrappers right before calling `update_protocol_yield_fee()` via the offchain code.

Jupiter: Can't be fixed effectively, will be managed in ops.

Cantina Managed: Acknowledged.

3.3.4 The names of update fee functions and MAX fee constants are confusing

Severity: Informational.

Context: admin.rs#L37-L43, admin.rs#L133-L139.

Description: The names of update fee functions and MAX fee constants are confusing.

For example `update_protocol_yield_fee()` and `MAX_YIELD_FEE_BPS` don't belong to eachother.

	maintenance	partner
MAX	MAX_MAINTENANCE_FEE_BPS	MAX_YIELD_FEE_BPS

update function	update_protocol_yield_fee	update_partner_fee

Recommendation: Consider changing the names in the following way:

```
- MAX_YIELD_FEE_BPS
+ MAX_PARTNER_FEE_BPS
```

```
- update_protocol_yield_fee()
+ update_protocol_maintenance_fee()
```

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.5 Function `transfer_protocol_authority()` isn't obvious

Severity: Informational.

Context: admin.rs#L58-L69.

Description: As explained by the protocol: rotation is deliberately two transactions:

- 1 upgrade the program with the new `GOVERNANCE_MS` constant,
- 2 call `transfer_protocol_authority(NEW)` signed by the old authority, which moves the stored authority to match.

Ordering is forced by the guard, since pre-upgrade the instruction can only accept the old constant. Two txs rather than one is intentional: the upgrade alone doesn't hand over control, and the handover requires the outgoing authority to still be live and signing.

However this isn't obvious when reading the code.

Recommendation: Consider adding a comment explaining this approach.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.6 The ProtocolConfig PDA is not always enforced

Severity: Informational.

Context: context.rs#L55-L59, context.rs#L72-L75, context.rs#L166-L173, context.rs#L313-L317, context.rs#L409-L421, context.rs#L504-L517, context.rs#L605-L619, context.rs#L715-L728.

Description: The following functions don't enforce the ProtocolConfig PDA:

- InitWrapper.
- Accrue.
- ClaimPartnerFees.
- DepositFToken.
- WithdrawToFToken.
- UnderlyingOp.

For comparison the following functions do have this:

- UpdateConfig.
- ClaimProtocolFees.
- UpdatePartnerFee.

This is inconsistent and could be an issue if a future upgrade accidentally creates a second valid `ProtocolConfig` account at a non-PDA address, for example during a migration.

Recommendation: Consider changing it in the following way:

```
  #[account(
+     seeds = [PROTOCOL_CONFIG_SEED],
+     bump = protocol_config.bump,
      constraint = protocol_config.authority == GOVERNANCE_MS @
       ↪  ErrorCodes::OnlyAuthority,
  )]
```

Jupiter: Acknowledged, no code change. Every affected instruction (`InitWrapper` , `Accrue` , `ClaimPartnerFees` , `DepositFToken` , `RedeemToFToken` , `UnderlyingOp`) each requires `protocol_config.authority == GOVERNANCE_MS` . `ProtocolConfig` has exactly one creation path in the program, `InitConfig` , which is itself PDA-derived (`seeds = [PROTOCOL_CONFIG_SEED]`) and gated behind the program's upgrade authority.

There is no second init path and no migration instruction, so no non-PDA account can ever satisfy the authority check.

Cantina Managed: Acknowledged.

3.3.7 Two different ways to create errors

Severity: Informational.

Context: admin.rs#L44, admin.rs#L68.

Description: There are two different ways used to create errors: with and without `.into()` . The `.into()` version drops the `error_origin` (file/line) metadata that `error!()` attaches.

Recommendation: Consider using the version without `.into()` :

```
- Err(ErrorCodes::...into());
+ Err(error!(ErrorCodes::..));
```

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.8 No explicit checks are done on `mints`

Severity: Informational.

Context: admin.rs#L83.

Description: No explicit checks are done on `mints` . The underlying `fluid` protocol does do this via `check_for_token_extensions_only_mint()` .

Recommendation: If ever another underlying protocol would be used, it would be useful to check, comparable to the way `fluid` does it, for example like this:

```
check_for_token_extensions_only_mint(&ctx.accounts.f_token_mint)?;
check_for_token_extensions_only_mint(&ctx.accounts.underlying_mint)?;
```

Jupiter: Since the underlying lending has these exact checks, there is no reason to add it here. If we list any token-22 extension on lending, we make sure to check everything beforehand with the token team.

Cantina Managed: Acknowledged.

3.3.9 Explicit checks on `fee_recipient`

Severity: Informational.

Context: admin.rs#L83-L86, admin.rs#L163-L165.

Description: Functions `init_wrapper()` and `update_partner_fee_recipient()` accept the partner fee recipient as an unchecked `Pubkey`. The program stores that address without verifying, at configuration time, that it is an existing token account for this wrapper's `fToken` `mint` and selected token program.

The address is validated only later in `claim_partner_fees()`. Therefore a typo, default key, uninitialized account, or account for the wrong mint is accepted during configuration and causes claims to fail only after fees have accrued.

Recommendation: Accept the recipient as an `InterfaceAccount<TokenAccount>` account and constrain it to the wrapper's `fToken` `mint` and token program:

```
#[derive(Accounts)]
  pub struct UpdatePartnerFeeRecipient<'info> {
      pub signer: Signer<'info>,

      #[account(
          mut,
          constraint = wrapper_config.authority == signer.key() @
           ↪  ErrorCodes::OnlyAuthority,
      )]
      pub wrapper_config: Box<Account<'info, WrapperConfig>>,

      #[account(
          address = wrapper_config.f_token_mint @
           ↪  ErrorCodes::WrapperFTokenMintMismatch,
      )]
      pub f_token_mint: Box<InterfaceAccount<'info, Mint>>,

      #[account(
          token::mint = f_token_mint,
          token::token_program = token_program,
      )]
      pub new_partner_fee_recipient: Box<InterfaceAccount<'info, TokenAccount>>,

      pub token_program: Interface<'info, TokenInterface>,
  }
```

Then store:

```
ctx.accounts.wrapper_config.partner_fee_recipient =
   ctx.accounts.new_partner_fee_recipient.key();
```

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.10 Restrictions to `ATA` not done consistently

Severity: Informational.

Context: context.rs#L66, context.rs#L94-L101, context.rs#L504, context.rs#L531-L536, context.rs#L606, context.rs#L633-L638, context.rs#L641-L646, context.rs#L715, context.rs#L740-L746, context.rs#L753-L759, context.rs#L762-L768.

Description: Accounts are sometimes restricted with `associated_token::`, which means it can only be an `ATA`. This is sometimes combined with `init_if_needed` to create the `ATA`.

Sometimes they are restricted with `token::`, which means any account from the right owner can be used, however this doesn't have to be an `ATA`. On L94 and L762 there is a comment about `ATA`, but in these instances an `ATA` isn't required.

Recommendation: Doublecheck where an `ATA` should be required and where not. If you want to restrict the use to `ATA`s make the following change. Be careful with situations where `init_if_needed` is relevant.

```
#[account(
      mut,
 -    token::mint = f_token_mint,
 -    token::authority = protocol_config.authority,
 -    token::token_program = token_program,
 +    associated_token::mint = f_token_mint,
 +    associated_token::authority = protocol_config.authority,
 +    associated_token::token_program = token_program,
  )]
```

Jupiter: Acknowledged. Commit 7be5e230 removes unused account constraints from several instructions (`ClaimProtocolFees`, `InitWrapper`, `Accrue`, `ClaimPartnerFees`, and `RedeemToFToken`). One inconsistency remains in `ClaimProtocolFees`: `protocol_authority_ata` is checked with `token::mint` and `token::authority`, so any suitable token account owned by the protocol authority is accepted (not strictly an ATA).

Cantina Managed: Acknowledged.

3.3.11 The old fee recipient doesn't get the fees at `update_partner_fee_recipient()`,

Severity: Informational.

Context: admin.rs#L163, admin.rs#L183.

Description: When doing `update_partner_fee_recipient()`, the old fee recipient doesn't get the fees until that moment. If its the same party and just a new address it doesn't matter though.

Note: the same situation occurs with `update_claim_authority()`, however that will be very unlikely to be a different party.

Recommendation: If relevant consider calling `claim_partner_fees()` to give the fees to the old fee recipient.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.12 Duplicate unused functions

Severity: Informational.

Context: lending.rs#L16-L37, lending.rs#L60-L69, lending.rs#L120-L130.

Description: The functions `build_update_rate_accounts()` and `update_rate()` of `LendingCpiAccounts` are duplicates of the same functions in `LendingUpdateRateCpiAccounts`, however the functions of `LendingCpiAccounts` are not used.

Recommendation: Consider removing the functions `build_update_rate_accounts()` and `update_rate()` of `LendingCpiAccounts`.

Jupiter: Fixed in PR 6.

Cantina Managed: Fix verified.

3.3.13 Checks that `mint` and `f_token_mint` are equal to their stored values

Severity: Informational.

Context: context.rs#L66, context.rs#L103-L107, context.rs#L303, context.rs#L320-L324, context.rs#L357, context.rs#L372-L376, context.rs#L409, context.rs#L440-L444, context.rs#L504, context.rs#L548-L552, context.rs#L606, context.rs#L649-L653, context.rs#L715, context.rs#L771-L778.

Description: The underlying protocol fluid does an extra check that both `mint` and `f_token_mint` are equal to their stored values. This protocol doesn't do that, which is fine as long as the underlying protocol does it.

Recommendation: If the underlying protocol would ever be changed it would be useful to add this check too:

```
-       #[account(mut)]
+       #[account(
+           mut,
+           has_one = mint @ ErrorCodes::WrapperFTokenMintMismatch,
+           has_one = f_token_mint @ ErrorCodes::WrapperFTokenMintMismatch,
+       )]
    pub lending: Box<Account<'info, Lending>>,

-       #[account(address = lending.mint @ ErrorCodes::WrapperFTokenMintMismatch)]
+       #[account()]
    pub mint: Box<InterfaceAccount<'info, Mint>>,
```

In struct: UnderlyingOp (context.rs:770-781).

```
        #[account(
            mut,
            address = wrapper_config.lending @ ErrorCodes::WrapperFTokenMintMismatch,
+           has_one = mint @ ErrorCodes::WrapperFTokenMintMismatch,
+           has_one = f_token_mint @ ErrorCodes::WrapperFTokenMintMismatch,
        )]
        pub lending: Box<Account<'info, Lending>>,
    //...
-       #[account(address = lending.mint @ ErrorCodes::WrapperFTokenMintMismatch)]
+       #[account()]
        pub mint: Box<InterfaceAccount<'info, Mint>>,
```

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.14 Unnecessary `mut` s

Severity: Informational.

Context: context.rs#L66, context.rs#L109-L114, context.rs#L303, context.rs#L326-L331, context.rs#L357, context.rs#L378-L383, context.rs#L409, context.rs#L446-L451, context.rs#L504, context.rs#L554-L559, context.rs#L606, context.rs#L655-L660.

Description: `mut` is used in several locations where it is not required because these accounts are not written by this instruction or its downstream CPI, which results in unnecessary exclusive transaction locks.

Recommendation: Consider removing the unnecessary `mut` s.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.15 Dead code

Severity: Informational.

Context: context.rs#L124-L125, context.rs#L252, context.rs#L358, context.rs#L461, context.rs#L670.

Description: Some code isn't used.

Recommendation: Doublecheck if it should have been used, or is used elsewhere. If not consider removing the code.

Jupiter: Fixed in commit 7be5e23.

Cantina Managed: Fix verified.

3.3.16 Frontrunning creating `ATA`

Severity: Informational.

Context: context.rs#L167, context.rs#L207-L215.

Description: In theory someone can frontrun the transaction and already create the `ATA`.

Trying to reuse that leads to other complications. So better is to call `InitWrapper` again with a newly signed transaction. The probability can be reduced via a private path.

Recommendation: If this happens in practice, submit `init_wrapper` via a private path (RPC).

Jupiter: The reason is fail-closed accounting. Partner-fee balance is a residual of `vault.amount` against tracked user/protocol balances, so under `init_if_needed` a pre-funded vault ATA doesn't revert—the wrapper is created with that balance already claimable as partner fees, at zero `xToken` supply and zero accrued yield. We'd rather the deploy fail than start a wrapper in that state. Griefing cost to us is a fresh `x_token_mint` keypair and a resubmit.

Cantina Managed: Acknowledged.

3.3.17 Funds can be stuck in transit ATA

Severity: Informational.

Context: context.rs#L217-L225.

Description: If someone accidentally sends funds to the transit ATA it will be stuck.

Recommendation: If you want to retrieve them, consider adding a function to do that.

Jupiter: We can later, if needed, upgrade the program and add a function to rescue funds.

Cantina Managed: Acknowledged.

3.3.18 No way to update the `name` / `symbol` / `uri`

Severity: Informational.

Context: context.rs#L256-L260.

Description: There is no way to update the `name` / `symbol` / `uri`. This might be useful to have, especially for the uri.

Recommendation: Consider adding functions to update these values.

Jupiter: Can be done later via upgrade if needed.

Cantina Managed: Acknowledged.

3.3.19 Functions `transfer_vault_to_partner_recipient()` and `transfer_vault_to_protocol_authority()` are very similar

Severity: Informational.

Context: context.rs#L140-L162, context.rs#L477-L499.

Description: The functions `transfer_vault_to_partner_recipient()` and `transfer_vault_to_protocol_authority()` are very similar. Keeping two similar functions the same makes maintenance more difficult.

Recommendation: Consider merging them and supplying the "to" as a parameter.

Jupiter: Fixed in commit f056d2e.

Cantina Managed: Commit f056d2e creates a shared `transfer_vault_f_tokens()` helper in `helper.rs` that both `ClaimProtocolFees` and `ClaimPartnerFees` now delegate to.

3.3.20 Names are not directly clear in struct `UnderlyingOp`

Severity: Informational.

Context: context.rs#L715, context.rs#L775, context.rs#L792, context.rs#L800, context.rs#L806.

Description: Several names are not directly clear in struct `UnderlyingOp`.

Recommendation: Consider renaming them in the following way:

- Lending ⇒ lending_market.
- Rate_model ⇒ liquidity_rate_model.
- Liquidity ⇒ liquidity_state.
- Rewards_rate_model ⇒lending_rewards_rate_model.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.21 6 data accounts are not explictly checked

Severity: Informational.

Context: context.rs#L715, context.rs#L789-L800, context.rs#L806-L809.

Description: 6 data accounts are not explictly checked, so malformed or misrouted dependency accounts are only detected later on.

Recommendation: This could be checked earlier in the following way:

```
-  use lending::state::Lending;
+  use lending::state::{Lending, LendingAdmin};
+  use lending_reward_rate_model::state::LendingRewardsRateModel;
+  use liquidity::state::{Liquidity, RateModel, UserSupplyPosition};

  pub struct UnderlyingOp<'info> {
  //...
-       #[account(mut)]
```

```
-       pub lending_supply_position_on_liquidity: UncheckedAccount<'info>,
+       pub lending_supply_position_on_liquidity: AccountLoader<'info,
↪  UserSupplyPosition>,
  //...
-       pub rate_model: UncheckedAccount<'info>,
+       pub rate_model: AccountLoader<'info, RateModel>,
  //...
-       #[account(mut)]
-       pub liquidity_vault: UncheckedAccount<'info>,
+       #[account(mut, associated_token::mint = mint, associated_token::authority =
↪  liquidity)]
+       pub liquidity_vault: Box<InterfaceAccount<'info, TokenAccount>>,
  //...
-       #[account(mut)]
-       pub liquidity: UncheckedAccount<'info>,
+       #[account(mut)]
+       pub liquidity: Box<Account<'info, Liquidity>>,
  //...
        pub liquidity_program: UncheckedAccount<'info>, // program ID
  //...
-       pub rewards_rate_model: UncheckedAccount<'info>,
+       pub rewards_rate_model: Box<Account<'info, LendingRewardsRateModel>>,
  //...
-       pub lending_admin: UncheckedAccount<'info>,
+       #[account(has_one = liquidity_program)]
+       pub lending_admin: Box<Account<'info, LendingAdmin>>,
  //...
  }
```

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.22 7 copies of get_lending_update_rate_cpi_accounts

Severity: Informational.

Context: context.rs#L129, context.rs#L342, context.rs#L394, context.rs#L466, context.rs#L574, context.rs#L675, context.rs#L821.

Description: There are 7 copies of `get_lending_update_rate_cpi_accounts()`, which makes maintenance more difficult.

Recommendation: Consider using a macro to reduce this to one copy.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.23 `*_SEED` in different files

Severity: Informational.

Context: constant.rs#L9-L10, seeds.rs#L1-L2.

Description: There are `*_SEED`s defined in different files. It might be more logical to put them in one place.

Recommendation: Consider moving all the `*_SEED` definitions to one file.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.24 The `EXCHANGE_PRICES_PRECISION` is hardcoded

Severity: Informational.

Context: constant.rs#L6-L7.

Description: The `EXCHANGE_PRICES_PRECISION` is hardcoded. It could be retrieved from the fluid protocol so make sure they stay in sync.

Recommendation: Consider using `lending::constant::EXCHANGE_PRICES_PRECISION`.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.25 `GOVERNANCE_MS` is the same as in the underlying `fluid` protocol

Severity: Informational.

Context: constant.rs#L12.

Description: The value of the `GOVERNANCE_MS` is the same as in the underlying `fluid` protocol.

Recommendation: Doublecheck they are intended to be the same value. If so consider using `lending::constant::GOVERNANCE_MS`.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.26 Meaning of `user_f_token_balance` and `protocol_f_token_balance` is not clear

Severity: Informational.

Context: state.rs#L18-L21.

Description: The name `user_f_token_balance` is confusing, it seems to be the balance for one user, however it is the sum of all user balances. The difference between `user_f_token_balance` and `protocol_f_token_balance` is also not obvious.

Recommendation: Consider renaming them in the following way:

- `user_f_token_balance` ⇒ `total_user_f_token_balance`.
- `protocol_f_token_balance` ⟹ `accrued_protocol_fee_f_tokens`.

Jupiter: Fixed in commit d0a5ac7e.

Cantina Managed: Fix verified.

3.3.27 Variable `last_f_token_price` has no comment about scaling

Severity: Informational.

Context: state.rs#L22.

Description: The variable `last_f_token_price` has no comment about scaling.

Recommendation: Consider the scaling as a comment, which most likely is `×EXCHANGE_PRICES_PRECISION`.

Jupiter: Fixed in commit d0a5ac7e.

Cantina Managed: Fix verified.

3.3.28 `partner_f_token_balance()` and `estimatePartnerClaimable()` behave differently

Severity: Informational.

Context: state.rs#L46-L52.

Description: Function `partner_f_token_balance()` reverts when `tracked > vault_scaled`. However the typescript function `estimatePartnerClaimable()` returns 0.

This seems inconsistent - and it might be useful to detect this situation from the typescript side, because its useful to detect this early.

Recommendation: Make sure the typescript function behaves the same as the onchain function.

Jupiter: Fixed in commit d0a5ac7e.

Cantina Managed: Fix verified.

3.3.29 Error `WrapperVaultBalanceUnderrun` not used

Severity: Informational.

Context: errors.rs#L27-L28, state.rs#L46-L52, math_test.rs#L337-L348.

Description: The error `WrapperVaultBalanceUnderrun` is not used. However the test: `underrun_errors_when_tracked_exceeds_vault` has a comment saying "tracked > vault must error with WrapperVaultBalanceUnderrun".

Recommendation: Consider using the error here:

```
  pub fn partner_f_token_balance(&self, vault_amount: u64) -> Result<u64> {
  //...
      let partner_scaled = vault_scaled
-         .safe_sub(tracked)?;
+         .safe_sub(tracked)
+         .map_err(|_| error!(ErrorCodes::WrapperVaultBalanceUnderrun))?;
  //...
  }
```

Jupiter: Fixed in commit e3d4065.

Cantina Managed: Fix verified.

3.3.30 Both error ranges overlap

Severity: Informational.

Context: errors.rs#L5-L8, errors.rs#L3-L6.

Description: When you define an `enum` with the `error_code` attribute, `Anchor` automatically assigns an error code starting from `6000`. However both the library and the main code use the same approach so the error messages will overlap.

Recommendation: Change one of the error ranges to a different offset:

```
-   #[error_code]
+   #[error_code(offset = 7000)]
  pub enum ErrorCodes {
  //...
  }
```

Jupiter: Fixed in commit e3d4065.

Cantina Managed: Fix verified.

3.3.31 Parameters for `pub struct LogAccrue` are confusing

Severity: Informational.

Context: events.rs#L68-L75, deposit.rs#L66-L72.

Description: The parameters for `pub struct LogAccrue` are confusing. The scaled versions are emitted but the parameters seem to indicate that the unscaled versions are emitted.

Recommendation: Consider changing the parameters to:

```
  #[event]
  pub struct LogAccrue {
  //...
-     pub total_delta: u128,
+     pub total_delta_scaled: u128,
-     pub protocol_delta: u128,
+     pub protocol_delta_scaled: u128,
-     pub partner_delta: u128,
+     pub partner_delta_scaled: u128,
  }
```

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.32 The comment for function `calc_total_delta()` is different than the implementation

Severity: Informational.

Context: helper.rs#L24-L41, fees.ts#L8-L20.

Description: The comment for function `calc_total_delta()` is different than the implementation. The comment uses one floor, while the code used two floors.

The typescript implementation of function `calcTotalDelta()` is the same as the comment of `calc_total_delta()`, but its different than the implementation of `calc_total_delta()`.

Recommendation: Consider changing the comment to:

```
- // total_delta_scaled = floor(      user_f_balance_scaled * partner_fee_bps * (fp -
↪  last) / (FEE_PRECISION  *              fp))
+ // total_delta_scaled = floor(floor(user_f_balance_scaled * partner_fee_bps
↪  /  FEE_PRECISION) * (fp - last) / fp)
```

Change the typescript implementation to also use the correct formula.

Jupiter: Fixed in commit 2f64862d.

Cantina Managed: Fix verified.

3.3.33 `partner_delta_scaled` has different meaning in different functions

Severity: Informational.

Context: helper.rs#L25, helper.rs#L36, deposit.rs#L19, deposit.rs#L57.

Description: The variable `partner_delta_scaled` has different meaning in different functions.

Recommendation: Consider renamning the variables to make the code clearer:

- `calc_total_delta()` : partner_delta_scaled ⟹ fee_rate_adjusted_balance_scaled.
- `accrue()` : partner_delta_scaled ⟹ net_partner_fee_scaled.

Jupiter: Fixed in commit d0a5ac7e.

Cantina Managed: Fix verified.

3.3.34 Failing `update_rate()` blocks all functions

Severity: Informational.

Context: deposit.rs#L19-L24.

Description: If `update_rate()` fails, for example when the underlying protocol is compromised, all the functions that call `accrue()` stop working. Users then are unable to retrieve their funds.

Recommendation: Consider having an emergency function (governance-activated), that lets holders burn `xTokens` for a share of the real `fToken` vault balance.

Jupiter: Acknowledged. If `update_rate()` fails, every instruction gated behind `accrue()` including redemptions halts, and user funds become temporarily inaccessible even though they remain fully backed by real `fTokens` in the vault.

We're not shipping a bypass-redeem path in this release. The program remains upgradeable under governance, and if this scenario materializes (or is prioritized in a future review), we'll add an instruction that pays out pro-rata against the vault's actual `fToken` balance without requiring a price refresh.

Cantina Managed: Acknowledged.

3.3.35 Pause functionality

Severity: Informational.

Context: deposit.rs#L19-L24.

Description: The `lend-referral` protocol doesn't have a pause functionality. However the underlying protocol Fluid does have it: here and here.

In case of an issue a pause of the underlying protocol blocks several functions, with some exceptions:

- `lending::update_rate()` keeps working, so `accrue()` also keeps working.
- `deposit_f_token()` and `withdraw_to_f_token()` never call `lending::deposit/withdraw()` so they keep working.
- `claim_partner_fees()` and `claim_protocol_fees()` also keep working.

Recommendation: Consider adding a pause functionality in case of issues/bugs in the software.

Jupiter: Fixed in PR 6.

Cantina Managed: PR 6 adds a `paused` flag to `ProtocolConfig`. When paused, deposit and withdrawal operations are blocked. Note that `update_partner_fee()` can still run while paused, but governance controls both pause and unpause.

3.3.36 Calculation of `maintainer_fee`

Severity: Informational.

Context: state.rs#L17, state.rs#L93, deposit.rs#L37-L42, deposit.rs#L50-L55.

Description: The `maintainer_fee` is a percentage of the `partner_fee`. This is not clearly documented in the code although it is implemented this way. This makes maintaining and reviewing the code more difficult.

Recommendation: Consider documenting this approach.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.37 No zero check `x_to_mint` in `mint_underlying()`

Severity: Informational.

Context: deposit.rs#L214-L218.

Description: Function `mint_underlying()` doesn't check for `x_to_mint` being 0. If `x_to_mint == 0` the transaction will succeed however no tokens are moved.

Recommendation: Consider checking for `x_to_mint == 0`.

Jupiter: Fixed in commit 1df2789.

Cantina Managed: Fix verified.

3.3.38 Inversion of the `fToken` price not 100% accurate in `mint_underlying()`

Severity: Informational.

Context: deposit.rs#L214, deposit.rs#L244-L249, deposit.rs#L263-L273.

Description: Function `mint_underlying()` currently uses a straightforward calculation to calculate `underlying_needed`. However because the deposit in the `fluid` protocol passes through three compounding liquidity-layer floors, this calculation isn't 100% accurate.

This means the function will revert most of the time when using realistic values.

Recommendation: The solution is to round up at every downstream floor so `deposit mints` ≥ `f_needed`.

This can be done in the following way:

```
+   use anchor_lang::prelude::AccountLoader;
+   use liquidity::state::TokenReserve;
  //...
  pub fn mint_underlying(...) ... {
  //...
-       let underlying_needed = safe_multiply_divide_up(
-           f_needed.cast::<u128>()?,
-           final_price,
-           EXCHANGE_PRICES_PRECISION,
-       )?
-       .cast::<u64>()?;
+       // Liquidity supply price L the deposit CPI will use.
+       let l = {
+           let loader = AccountLoader::<TokenReserve>::try_from(
+               &ctx.accounts.supply_token_reserves_liquidity.to_account_info(),
+           )?;
+           let (supply_exchange_price, _) =
 ↪  loader.load()?.calculate_exchange_prices()?;
+           supply_exchange_price.cast::<u128>()?
+       };

+       let q = EXCHANGE_PRICES_PRECISION;
+       let registered_target = safe_multiply_divide_up(f_needed.cast::<u128>()?,
 ↪  final_price, q)?;
+       let raw_target = safe_multiply_divide_up(registered_target, q, l)?;
```

```
+      let underlying_needed = safe_multiply_divide_up(raw_target, 1,
↪  q)?.cast::<u64>()?;
  //...
  }
```

Jupiter: This is the fix commit 83965b7 which addressed this issue.

Cantina Managed: Fix verified.

3.3.39 Name `withdraw_to_f_token()` not consistent

Severity: Informational.

Context: lib.rs#L115, lib.rs#L119, user.rs#L49, user.rs#L53, withdraw.rs#L18, withdraw.rs#L80.

Description: Function `withdraw_to_f_token()` uses `x_shares` as input. It would be more logical if this was called `redeem_to_f_token()`, similar to `redeem_to_underlying()` which also uses `x_shares` as input.

Recommendation: Consider doing the following renames:

- `withdraw_to_f_token()` ⇒ `redeem_to_f_token()`.
- `withdraw_to_f_token_with_min_out()` ⇒ `redeem_to_f_token_with_min_out()`.

Jupiter: Fixed in commit e3d4065.

Cantina Managed: Fix verified.

3.3.40 `x_supply` could be 0

Severity: Informational.

Context: withdraw.rs#L18, withdraw.rs#L35-L39, withdraw.rs#L80, withdraw.rs#L94-L98.

Description: In functions `withdraw_to_f_token()` and `redeem_to_underlying()`, `x_supply` could be 0, for example when a new wrapper is deployed. In that case the divide will error.

This could be detected to show a more specific error.

Recommendation: Consider checking for `x_supply == 0` and giving a specific error.

Jupiter: Fixed in commit 5ee4bb6](commit 5ee4bb6d).

Cantina Managed: Fix verified.

3.3.41 Comment in function `div()` is not accurate

Severity: Informational.

Context: u256.rs#L325-L326, u256.rs#L333-L335.

Description: Comment in function `div()` is not accurate: the function panics when dividing by 0.

Recommendation: Consider updating the comment.

Jupiter: Acknowledged.

Cantina Managed: Acknowledged.

3.3.42 Calculation in test `ftokens_burned_for_withdraw()` is incorrect

Severity: Informational.

Context: withdraw.rs#L149, withdraw.rs#L165-L171, math_test.rs#L781, math_test.rs#L792-L798.

Description: The calculation in test `ftokens_burned_for_withdraw()` is incorrect:

- `ftokens_burned_for_withdraw()` computes: floor(underlying_out · price / Q).
- `withdraw_to_underlying()` computes: ceil(underlying_out · Q / price).

Recommendation: Consider using the same calculation, for example by moving the calculation to a function in `helper.rs` and call it from the testscript.

Jupiter: Fixed in commit d0a5ac7e.

Cantina Managed: Fix verified.

3.3.43 Accrual events omit wrapper and token mint

Severity: Informational.

Context: deposit.rs#L66-L72.

Description: `LogAccrue` does not include the identity of the wrapper or fToken mint whose fee buckets were updated. The event contains only the old and new Lending prices and the total, protocol, and partner deltas:

```
pub struct LogAccrue {
    pub old_price: u128,
    pub new_price: u128,
    pub total_delta: u128,
    pub protocol_delta: u128,
    pub partner_delta: u128,
}
```

The same referral program can operate multiple wrappers. Two wrappers can therefore emit identical `LogAccrue` payloads for the same price movement and fee configuration, while their `wrapper_config`, fToken mint, balances, and recipients are different. An event consumer using the event payload alone cannot determine which wrapper's accounting state changed.

Recommendation: Add `wrapper_config` and, preferably, the fToken mint to `LogAccrue`.

Jupiter: Fixed in commit 2f64862.

Cantina Managed: Fix verified.